

CSC 330 Programming Languages

Lecture #2

Serialization and Multi Language Communication

Now that you understand how different languages handle the creation of user defined data types (UDTs), let's look at how different languages can communicate. Believe it or not, code written in one language can communicate with code written in another language. For example, the game engine Unity is mostly written in C++ for the core engine features but there is a layer of C# written on top. It is common for webpages to have Javascript send messages to server-side code written in PHP. The PHP code can then send messages back to Javascript. The kernel of your operating system is probably written in C, yet you can make system calls from a driver written in Rust. How is it possible for a language to invoke code written in a different language? There are multiple ways of accomplishing this.

Serialization and Message Passing

To get two languages to communicate, there must be an intermediate format for the message being passed. Javascript doesn't understand PHP, C doesn't understand C#, Java doesn't understand Python. But if we agree upon a separate representation of data, then language 1 can write its message in this representation and language 2 can read it. The message could be a function to invoke or even an entire data object. It could also be parameter information for a function call.

There are several types of intermediate message representations. We will need to use serialization for this. The process of "serialization" just means to convert something into a string representation. We can convert anything into a string, whether that be a string of characters or a string of bits. Once we have a string, we can send it over as a message to another language (or another machine).

XML

XML stands for eXtensible Markup Language. It is similar to HTML in that it uses tags as a way of delimiting data. For example, given the following class written in Java:

```
public class Data {
    public int number;
    public String name;
    public boolean hasData;

    public Data(int number, String name, boolean hasData) {
        this.number = number;
        this.name = name;
        this.hasData = hasData;
    }
}
```

And given the following object created from this class:

```
Data data = new Data(42, "info", true);
```

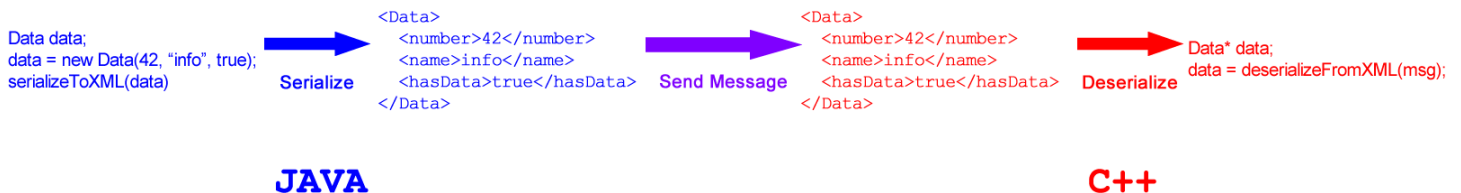
The following would be the representation of this object in XML:

```
<Data>
  <number>42</number>
  <name>info</name>
  <hasData>true</hasData>
</Data>
```

We could then use this XML data to essentially send the object from one language to another. One quick way of doing this is through serialization libraries. This will allow us to just specify the Data class and then use the library to create the XML from an object of that class. The receiving language would then deserialize the object. This assumes that we have the same class representation in both languages. Let's say that we want Java to communicate with C++, first we would have to have the same class in both languages:

Java	C++
<pre> public class Data { public int number; public String name; public boolean hasData; public Data(int number, String name, boolean hasData) { this.number = number; this.name = name; this.hasData = hasData; } } </pre>	<pre> #include <string> class Data { public: int number; std::string name; bool hasData; Data(int number, std::string name, bool hasData) { this->number = number; this->name = name; this->hasData = hasData; } }; </pre>

Then we would create an object in Java, serialize it to an XML string, send over the string, then deserialize back into the original object but now C++ has the object:



This may seem like a lot of work but if you get the right libraries, it can actually be pretty straightforward. Plus, you can always create helper methods in both languages to make this process go smoother.

XML has been around for almost 30 years, and it's still being used today. One downside however is the extra information needed to send a message. If we look at the simple message above, we notice that 76 characters (not including spaces) are XML formatting while only 10 characters are actual values! Even if we include the names of the fields, that leaves 27 meaningful characters with 59 characters of XML. That's not a good ratio. This is why our next type of formatting is more preferred by some developers.

JSON

JSON stands for JavaScript Object Notation. Despite the name, it is used for more than just Javascript related tasks. You can pronounce it either like a person's name "Jason" or you can stress the J and say "san" like "Jay san". Either way, this is a nice formatting language for compactly representing data. Everything is represented in key value pairs like a dictionary. The Data object we had before would look like this in JSON:

```
{ "number":42, "name":"info", "hasData":true }
```

JSON only specifies the name of each field once, then follows it up with its value. It uses commas to delimit the key-value pairs and each dictionary starts and ends with braces. So, with 27 meaningful characters in the data, only 15 extra characters are required for the JSON format!

The process is the same for sending and receiving this data. Create the class in both languages, create an object in one language, serialize the object to a JSON string, send over the string, deserialize it with the other language and you're done.

How about for objects with composition? Both JSON and XML allow for nesting, so this isn't an issue. Given the following classes:

```
class Vec3 {
    public float x;
    public float y;
    public float z;

    public Vec3(float x, float y, float z) {
        this.x = x;
        this.y = y;
        this.z = z;
    }
}

class Line {
    public Vec3 pt1;
    public Vec3 pt2;

    public Line(Vec3 pt1, Vec3 pt2) {
        this.pt1 = pt1;
        this.pt2 = pt2;
    }
}
```

And an object:

```
Line line = new Line(new Vec3(0, 0, 0), new Vec3(1, 2, 3));
```

Here would be the JSON and XML for that object. Notice that (not counting any whitespace) the JSON version is 65 characters while the XML version is 83 characters.

JSON

```
{
  "pt1": {"x": 0.0, "y": 0.0, "z": 0.0},
  "pt2": {"x": 1.0, "y": 2.0, "z": 3.0}
}
```

XML

```
<Line>
  <pt1>
    <x>0</x>
    <y>0</y>
    <z>0</z>
  </pt1>
  <pt2>
    <x>1</x>
    <y>2</y>
    <z>3</z>
  </pt2>
</Line>
```

Binary

Sometimes you don't need the data format to be "human readable", or able to be easily understood by a human. Sometimes you would rather have faster serialization/deserialization and possibly smaller data size. For these purposes, you can use binary serialization. This is where the data is stored as a bit string. The serializer goes to each field and encodes it in a sequence of bits that make sense for that data type.

Since the data is stored as bits, looking at this data with a simple text editor is useless. Your text editor will assume the data is ASCII or Unicode and will try to display it as such, leading to a lot of strange characters.

You may think that this is the most secure form of serialization, but actually it isn't! If you receive a binary serialized file and you don't absolutely trust the sender, deserializing it can be as dangerous as running an executable from an untrustworthy source. In fact, Microsoft wrote an entire article about the dangers of binary serialization¹. If you are creating an app for yourself or building a standalone app to be used on a computer not connected to any network (or plenty of other use cases), this is probably fine to use as security isn't of the upmost importance in these scenarios. A bigger issue is compatibility. Since we are going down to the bit level, we may experience encoding/decoding differences among libraries, operating systems, or even architectures.

Delimited Text

If the messages you are passing are simple enough (for example, instead of sending an entire object of a class, you just want to send some values), you can send them in a delimited text format. You normally just use the text of the message with an agreed upon delimiter between the pieces of data (if there is more than one piece of data you are sending). This has obvious limitations, but for a small number of values or a large number of values that have the same data type, this is an acceptable approach. There are two standards here we can use:

Comma Separated Values (or CSV): 45,1,-5,7894

Tab Separated Values (or TSV): 45 1 -5 7894

We could also come up with our own delimiter, for example a colon: 45:1:-5:7894

As long both languages agree upon the delimiter, this can work just fine. Most languages contain a built-in function to split a string based on a given delimiter (e.g. ' ' or '\t' or ':') and will return an array or list of strings. So, the above message can easily be split into { '45', '1', '-5', '7894' }. After splitting, you can loop through and parse each string as an integer.

You can use multiple characters as a delimiter if you wish. If you are sending strings coming from the user, just think of a highly unlikely sequence of characters. For example, most non-developers don't use tilde or back tick. So, you could do the following:

```
user given string 1~`~user given string 2~`~user given string 3
```

Then just split on ~`~ and you can have an array of the individual strings. If you plain on looking at this message directly, you can add spaces if you wish to make it nicer to read:

```
user given string 1 ~`~ user given string 2 ~`~ user given string 3
```

¹ <https://learn.microsoft.com/en-us/dotnet/standard/serialization/binaryformatter-security-guide>

Just make sure to trim your strings after splitting them. For example, in Python this would be:

```
message = ... # receive full message
parts = message.split("~`~") # split based on delimiter
for part in parts:
    data = part.strip() # remove leading and trailing whitespace
    # do something with the data
```

Most languages have a function named `trim()`, or `strip()` or even `chomp()` that will get rid of white spaces before and after the string (but not inside the string).

Using a delimited text format is nice for simple messages as it has very little overhead. The only real overhead is the repeated use of a delimiter. However, as soon as you need any sort of hierarchy to your data, or key-value pairs, or multiple lines with different delimiters, you will probably want to switch to XML, JSON, or binary.

Live Demo

If I haven't already, I will now show an example of this live using C# to serialize an object into all three forms (XML, JSON, and binary) and then deserialize it using Python. All demos for this lesson are on Canvas for you to run locally.

Other Uses

There are multiple uses for serialization. I'm using the context of communication among different programming languages. However, you can also use serialization to remember the state of objects between application shut down and startup. This can be useful for saving data in a game, saving cached information, OS hibernation, and many other uses. It can also be useful if a particular object took a while to create. Think of doing complex math calculations to find a set of float values. Instead of performing those calculations each time the application is ran, you could perform them once, create an object with those float values, then serialize it to a file. Next time the application is executed, look for the file. If it exists, deserialize it from the file. You can also use serialization to pass information between different machines (like a client and a server). There are many use cases for this technique, and I personally have used this several times in industry (as well as personal) projects.

Multi Language Communication

You can use any of the above forms of serialization to pass messages back and forth among different machines and different languages. Here are some ways of achieving that communication.

Client-Server Model

AJAX is a popular tool used to submit data to a server and get a response. I've used it several times myself. In fact, if you have participated in the CSC 220 time complexity challenge, you have used a webpage that uses AJAX. The acronym stands for Asynchronous Javascript And XML. Essentially, you use Javascript to send a message written in XML to a server (running PHP, JSP, ASP, or some other language). The server-side language then sends Javascript a response back (that's also in XML). So, XML becomes the intermediate format that both languages agree to talk in. Using libraries such as jQuery² can make this process very easy.

² <https://api.jquery.com/jquery.ajax/#jQuery-ajax-url-settings>

SWIG

SWIG stands for Simplified Wrapper Interface Generator³. It allows you to write code in languages like C# and have it call functions from C++ code. This type of communication doesn't require XML or JSON, as you can directly call functions in the C++.

Say you have some C++ header files and corresponding source files. You simply create a file with extension ".i" (stands for "interface" file) that denotes all the C++ code you want to create wrappers for. The file is of the form:

```
/* File : filename.i */
//-----
%module module_name
//-----

/* References */
//-----
%{
    #include "reference_1.hpp"
    #include "reference_2.hpp"
}%
//-----

/* Code to convert */
//-----
// type definitions
typedef unsigned int uint32;
typedef unsigned char byte;

// input to file
#include "std_string.i"
#include "arrays_csharp.i"
%apply double INPUT[] {const double *P}

// modules to convert
#include "myfile_1.hpp"
#include "myfile_2.hpp"
//-----
```

Put this file in with your C++ code files and then run the following in the terminal:

```
swig -c++ -csharp -o wrapper.cpp interfacefile.i
```

³ <https://swig.org>

This will take your interface file and create wrappers for it. The first argument is the input language and the second is the language you want to use to communicate with it (i.e. in this example C++ is input and C# is the language I want to communicate with C++). The -o argument allows you to specify the name of the generated wrapper file. The generated wrapper file might look something like this:

```
/* -----
 * This file was automatically generated by SWIG (http://www.swig.org).
 * Version 3.0.2
 *
 * This file is not intended to be easily readable and contains a number of
 * coding conventions designed to improve portability and efficiency. Do not make
 * changes to this file unless you know what you are doing--modify the SWIG
 * interface file instead.
 * -----
 */

#define SWIGCSHARP

#ifdef __cplusplus
/* SwigValueWrapper is described in swig.swg */
template<typename T> class SwigValueWrapper {
    struct SwigMovePointer {
        T *ptr;
        SwigMovePointer(T *p) : ptr(p) { }
        ~SwigMovePointer() { delete ptr; }
        SwigMovePointer& operator=(SwigMovePointer& rhs) { T* oldptr = ptr; ptr = 0;
delete oldptr; ptr = rhs.ptr; rhs.ptr = 0; return *this; }
    } pointer;
    SwigValueWrapper& operator=(const SwigValueWrapper<T>& rhs);
    SwigValueWrapper(const SwigValueWrapper<T>& rhs);
public:
    SwigValueWrapper() : pointer(0) { }
    SwigValueWrapper& operator=(const T& t) { SwigMovePointer tmp(new T(t));
pointer = tmp; return *this; }
    operator T&() const { return *pointer.ptr; }
    T *operator&() { return pointer.ptr; }
};

template <typename T> T SwigValueInit() {
    return T();
}
#endif

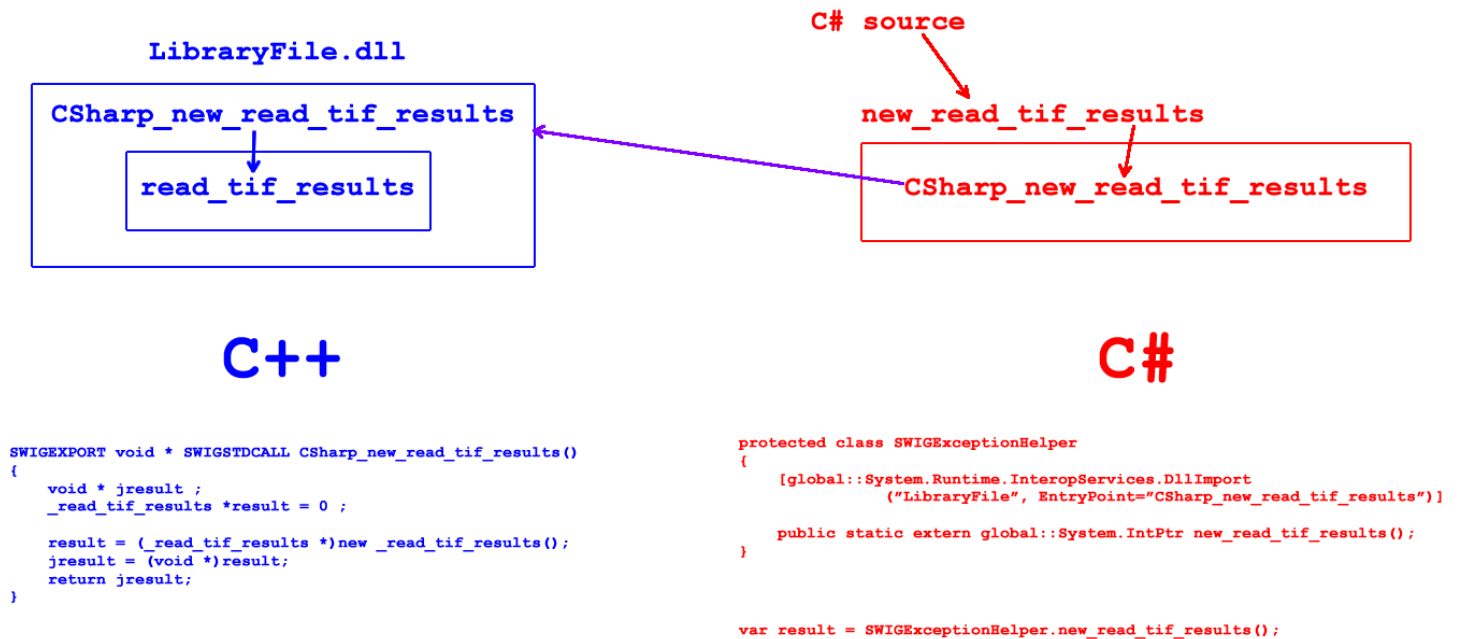
SWIGEXPORT void * SWIGSTDCALL CSharp_new_read_tif_results() {
    void * jresult ;
    _read_tif_results *result = 0 ;

    result = (_read_tif_results *)new _read_tif_results();
    jresult = (void *)result;
    return jresult;
}
```

Swig will also generate files for the other language as well. An example of a generated file could look like this:

```
/* -----  
 * This file was automatically generated by SWIG (http://www.swig.org).  
 * Version 3.0.2  
 *  
 * Do not make changes to this file unless you know what you are doing--modify  
 * the SWIG interface file instead.  
 * -----  
*/  
  
public class SWIGTYPE_p_int {  
    private global::System.Runtime.InteropServices.HandleRef swigCPtr;  
  
    internal SWIGTYPE_p_int(global::System.IntPtr cPtr, bool futureUse) {  
        swigCPtr = new global::System.Runtime.InteropServices.HandleRef(this, cPtr);  
    }  
  
    protected SWIGTYPE_p_int() {  
        swigCPtr = new global::System.Runtime.InteropServices.HandleRef(null,  
global::System.IntPtr.Zero);  
    }  
  
    internal static global::System.Runtime.InteropServices.HandleRef  
        getCPtr(SWIGTYPE_p_int obj) {  
        return (obj == null) ? new  
            global::System.Runtime.InteropServices.HandleRef(null,  
            global::System.IntPtr.Zero) : obj.swigCPtr;  
    }  
}  
  
class PINVOKE {  
    protected class SWIGExceptionHelper {  
        [global::System.Runtime.InteropServices.DllImport("LibraryFile",  
            EntryPoint="CSharp_new_read_tif_results")]  
        public static extern global::System.IntPtr new_read_tif_results();  
    }  
}
```

We can now build the C++ code into a library and use the C# wrapper to call functions inside of that library. Notice how in the C++ code we have a function called "CSharp_new_read_tif_results". This function was created by SWIG and essentially just calls "read_tif_results" from the C++ source code. This is compiled into a library (which I call "LibraryFile" for this example) and in the C# code notice the "DllImport" to a file called "LibraryFile" with the entry point set at the function "CSharp_new_read_tif_results". SWIG creates a C# function called "new_read_tif_results" that is mapped to this library function CSharp_new_read_tif_results. So, in my C# code I just need to call new_read_tif_results and it will essentially call the compiled C++ function. This diagram might help this make more sense:



Named Pipes

Any time a process wants to communicate with another process, we call this IPC which stands for Interprocess Communication. A common way of performing IPC is using named pipes. There needs to be a server and a client for this to work (but they may exist on the same machine), and each application opens a pipe/file for communication then gives that pipe/file a shared name. For example, the server might be created in C# with plenty of C# source code and a function to create a pipe with the name "my_pipe". The server will open this pipe and wait for anyone to connect to it. Another process (that may be written in C++ for example and contain plenty of C++ source code) will open up a pipe with the same name (i.e. "my_pipe"). The server will get alerted of this connection and will wait for any messages to be sent from the C++ application. At this point the C++ application can go about its business running any code and performing any actions from the user. Once the C++ application wishes to invoke the C# application and possibly run a function, it will send a message through this pipe to the C# server. This message could be in XML, JSON, or a custom text format. Remember that the C# server is stuck waiting on any incoming messages. The C++ side will send it a message and then wait for the result. The C# side will receive the message, do whatever it says (such as running one or more C# functions), get a result from that and send it back to the C++ application. The C++ application will receive the result and use it in whichever way it sees fit. The pipe is left open for further communication while the C++ application is running.

This is a more indirect way of communication since we aren't directly invoking a library function in another language. While that normally means the communication will be slower, I've found in practice that IPC via named pipes is plenty fast enough! Windows, Mac and Linux directly support this type of communication (with minor differences in implementation) and there is normally very little setup required to get this to work. With SWIG, more setup is required and there is generated code to compile and use. With named pipes, we can accomplish communication with a few lines of code from both applications. In the next programming assignment, you will be using named pipes to have a C# process communicate with both a C++ application and a Python application.

RPC

It is possible to directly call a method from a server using a client machine. This is known as a "remote procedure call" or RPC for short. Normally the server and client are two completely different machines and, again, the server and client do not have to use the same language. However, so we can see an example that

you can run yourself on your laptop, here is some Python code for both the server and client. You can run both of these on your laptop and have your laptop act as both the server and client. Here is the server code:

File: server.py

```
# import module used for RPC
import rpyc

# create a class that inherits from the RPC module's Service class
class MyService(rpyc.Service):
    # any function you want to remotely call must begin with the prefix "exposed_"
    def exposed_myMagicNumber(self):
        # have the function do whatever you want and return a value
        return 85

if __name__ == "__main__":
    # grab the ThreadedServer class
    from rpyc.utils.server import ThreadedServer

    # create a thread and give it the Service class above
    # pick any port. The client must use this port to communicate
    t = ThreadedServer(MyService, port=18861)

    # start the thread and wait for a request from a client
    t.start()
```

You will need to install the rpyc module to make this work. Just run "pip install rpyc" in the terminal to install it. If this doesn't work, try "python -m pip install rpyc". If using Mac or Linux, you may have to use "python3" as the command instead of "python". The code above is executed on the server, which causes the server to spawn a thread and then wait for communication from a client. Open a terminal (like Git Bash or Powershell), and run the server code using "python server.py" (or if using Mac/Linux, "python3 server.py"). You will notice the code will run but nothing happens. It just seems to hang there. That's fine, since it is waiting for a connection. Leave this window alone and let's move on to the client:

File: client.py

```
# import module used for RPC
import rpyc

# establish a connection
# Must use the IP of the server along with the port number
conn = rpyc.connect("localhost", 18861)

# use that connection to call our remote function
result = conn.root.exposed_myMagicNumber()

# print out what it returned
print("Result:", result)
```

With the server.py file running, open a new terminal. In this terminal, run the client.py file. You should see "85" printed in the terminal. The client code starts up and establishes a connection with the server using the server's IP (in this case we use "localhost" since it's the same machine) and the port number. It then invokes

the "exposed_myMagicNumber" function on the server, which actually runs on the server and returns the value 85. The client grabs this value and puts it on the screen. Pretty cool huh?

Going into this in detail is probably best left to a networking class (like CSC 450) but the point is that we can make two different machines communicate possibly using different languages for the client and server. In fact, here is an example of using different languages. You might not have C# on your machine so you may not be able to run this, but I'll show it in class. Here is an example of using RPC with Python as the server and C# as the client:

File: server.py

```
# import the stuff we need
from xmlrpc.server import SimpleXMLRPCServer

# Create server with port number
server = SimpleXMLRPCServer("", 18861)
server.register_introspection_functions()

# our RPC function
def MyMagicNumber():
    return 42

# register the RPC function so client can call it
server.register_function(MyMagicNumber, 'MyMagicNumber')

# Run the server thread
server.serve_forever()
```

This requires a different module that you can install using "pip install xmlrpcutils". After running this on the server, use this for the C# client:

File: client.cs

```
// import the stuff we need
using CookComputing.XmlRpc;
using System;

// create a stub that matches the server function we want to call
[XmlRpcUrl("http://localhost:18861")]
public interface IMyInterface : IXmlRpcProxy {
    [XmlRpcMethod]
    int MyMagicNumber(); // RPC function. Implement lives on server.
}

public class Program {
    public static void Main(string[] args) {
        // create rpc object
        IMyInterface proxy = XmlRpcProxyGen.Create<IMyInterface>();

        // call our RPC function
        Console.WriteLine(proxy.MyMagicNumber());
    }
}
```

You will need to install the "CookComputing.XmlRpcV2" dll for this to work. Running this while the server is running produces the magic number 42 in the terminal.

GRPC

RPC uses binary serialization for communication. It relies on the developer to add security measures on top of its normal use. More modern protocols such as GRPC allow for JSON and XML communication and have security measures built in. The discussion of GRPC is outside of the scope of this course, but I just wanted to mention that if you are going to use remote procedure call, you might want to use more modern technologies such as GRPC. I personally have used GRPC. The company I was working for was partnering with another company. This other company had some code written in Go and we used GRPC to get C# to call these Go functions. Actually, since GRPC uses HTTP 2.0 and since we were using the Unity game engine which (at the time of the project) didn't support HTTP 2.0, we ended up making a separate C# program that used GRPC to call the Go functions. We then used IPC to have the Unity project communicate with the stand alone C# project. So, Unity sent a message to the standalone app using IPC. The standalone app remotely called a Go function using GRPC. It got the result and then used IPC again to send the result back to Unity. A little messy, but sometimes solutions have to be! It's important to understand multiple communication methods incase some are just not supported or not desired to be used under the circumstances, and sometimes multiple methods are required. Even though in depth discussions on these other techniques (such as GRPC) is outside the scope of this course, it should be much easier to learn these now that you understand the basics.